Renode Lab on GitHub Codespaces

Nine labs from a 5-min ARM MMIO warm-up to a custom C# peripheral and optional multi-node IoT capstone — emulated entirely in your browser.

Welcome. Over the next few hours you will simulate nine different embedded systems — from a 5-minute MMIO taste on a bare-metal ARM Cortex-A9, through a Cortex-M4 microcontroller and a multi-core RISC-V Linux SoC, all the way to a SiFive FE310 with timer interrupts, a Robot-driven CI suite, a peripheral you write yourself in C#, and an optional multi-node IoT network — all inside a browser tab, with **nothing installed on your laptop**.

This document walks you through the setup once. After that, everything you need is in the per-lab `README.md` files in the repository.

1. What you need before starting

1.1 A personal GitHub account

If you don't already have one, create a **free personal account** at https://github.com/join. School-issued accounts work too, but a personal account is preferred so the Codespace and any edits you make follow you after the course ends.

GitHub's free tier includes:

Resource	Free quota / month
Codespaces compute	120 core-hours (≈ 60 hours on a 2-core machine)
Codespaces storage	15 GB-month

The whole renode-lab fits comfortably in those limits if you **stop your Codespace** when you take a break and **delete it** when you're done for the day. Leaving a Codespace running idle burns compute hours; leaving it created-but-stopped burns storage. We'll cover both at the end.

1.2 (Recommended) The GitHub Student Developer Pack

If you're a student, apply for the free **GitHub Student Developer Pack**. It upgrades your account to **GitHub Pro** at no cost, which **roughly doubles your Codespaces quota** — a real benefit for this lab and every future one you run in Codespaces:

Resource	Free account	With Student Pack (Pro)
Codespaces compute	120 core-hours / month	**180 core-hours / month**
Codespaces storage	15 GB-month	**20 GB-month**

(Quotas are GitHub's published figures at the time of writing and may change; check your account's Billing page for the current numbers.)

The Pack also bundles dozens of other free developer tools and cloud credits you'll find useful beyond this course.

How to get it (do this before your first launch if you can — approval can take anywhere from a few minutes to a few days):

1. Make sure you're signed in to the **personal GitHub account** from §1.1 (apply on the account you'll actually use for the labs).
2. Go to https://education.github.com/pack and click **Sign up for Student Developer Pack** (then **Get student benefits**).
3. Choose your account type as **Student** and select your school.
4. **Verify you're a student.** Fastest is a **school-issued email address** (`@school.edu`) — GitHub often approves it instantly. If you don't have one, upload **dated proof of enrollment** instead (student ID, class schedule, transcript, or enrollment letter with your name and school name visible).
5. Add your school name and **expected graduation date**, then submit.
6. Watch for the approval email. Once approved, **GitHub Pro is applied to your account automatically** — there's nothing to install, and your higher Codespaces quota takes effect immediately.

> **Already have it?** If your account already shows GitHub Pro (Student), you're done — just launch the lab in §2. **Not a student, or waiting on approval?** The free-tier quota in §1.1 is enough to complete every lab; the Pack simply gives you more headroom.

1.3 A modern web browser

Chrome, Firefox, Safari, or Edge — anything from the last two years. You will spend the entire lab in two browser tabs:

1. The **VS Code** tab that GitHub auto-opens (your terminal + editor live here).
2. The **noVNC desktop** tab on port 6080 (Renode's GUI analyzers — UART windows, GPIO/LED indicators, etc.).

1.4 Working knowledge of basic Linux

You should be comfortable doing all of these from a terminal without looking them up:

- Navigating: `pwd`, `cd`, `ls`, `ls -la`
- Reading files: `cat`, `less`, `head`, `tail`, `tail -f`
- Editing files: `nano`, `vim`, or just VS Code's editor pane
- Pipes and redirection: `|`, `>`, `>>`, `2>&1`
- Background jobs: `&`, `Ctrl-C`, `Ctrl-Z`, `jobs`, `fg`
- Inspecting processes: `ps`, `pgrep -af <name>`
- Reading exit codes and recognising errors

You do **not** need to know Docker, Kubernetes, or anything about Codespaces internals.

1.5 Familiarity with basic SoC concepts

You should already understand, at a hand-wave level:

- A CPU **fetches** instructions from memory addresses, then **executes** them. The address it's about to fetch from is the **program counter (PC)**.
- A **bus** lets the CPU talk to memory regions and **memory- mapped peripherals** (UART, GPIO, timers, interrupt controller, etc.). Each peripheral lives at a known base address.
- A **UART** sends bytes one wire at a time; on most cores there's a "transmit holding register" and a "line status register" with a `TX-FIFO-empty` bit.
- A **GPIO port** has registers for direction (input/output), output data (`ODR`), and atomic set/reset (`BSRR` on STM32).
- An **ELF** is a packaged binary with a known **entry point**; on Cortex-M, after reset the CPU loads SP from address `0x0` and PC from address `0x4` (the vector table).
- A **bootloader** (BBL, OpenSBI, U-Boot) initialises the SoC enough to hand control to a kernel.
- A **kernel** schedules processes; a **userspace shell** like BusyBox is the program you type commands into.

Lab 01 will exercise the first four bullets, lab 02 the last three, and lab 03 lets you build the bus + RAM + UART picture from nine lines of declarative platform description.

2. First launch (only do this once)

2.1 Open the lab repository

Visit https://github.com/prag79/renode-lab. Sign in to your GitHub account if prompted.

2.2 Click the "Open in GitHub Codespaces" badge

Near the top of the page you'll see this badge:

Open in GitHub Codespaces

Click it. GitHub will:

1. Show a **"Create codespace"** screen — accept the defaults (2-core machine, 8 GB RAM, 32 GB storage). Click **Create codespace**.
2. Spin up a small Linux VM somewhere in their cloud (≈30 s).
3. Pull the prebuilt image `ghcr.io/prag79/renode-lab:latest` from the GitHub Container Registry (≈30–60 s, only on first launch — afterwards it's cached).
4. Open VS Code in your browser, attached to the running container.

A second tab will auto-open at `*.app.github.dev:6080/vnc.html` — that's the noVNC desktop. If it shows "Failed to connect", give it 10 seconds and click **Connect** in the noVNC page.

2.3 Verify the environment

In the VS Code terminal (open one with **Ctrl-`** if it isn't already), type:

```
lab list
```

You should see all the available labs:

```
Available labs (00–07 core, 08 optional capstone):
  00       - bare-metal ARM Cortex-A9 + SmartTimer MMIO demo (also: lab demo)
  01       - bundled STM32F4 demo (sanity check)
  02       - Linux on SiFive HiFive Unleashed (RISC-V)
  03       - custom RV64 SoC + bare-metal hello world
  04       - bare-metal on a SiFive FE310 (HiFive1): UART + GPIO blink
  05       - FE310 timer interrupts: blink from a CLINT ISR
  06       - headless regression testing with the Robot framework
  07       - model your own peripheral (custom C# timer IP)
  08       - (optional) multi-node IoT network: 3x FE310 over a shared UART bus
  monitor  - plain Renode interactive monitor
```

Lab **08** is an optional capstone — three simulated FE310 boards on one shared UART bus (sensors + gateway). You don't need it for the sanity check below; run it after labs 00–07 when you're ready.

Then run lab 01:

```
lab 01
```

You should land at a `(STM32F4_Discovery)` prompt within a couple of seconds. That's the **Renode monitor** — Renode's interactive command line. From here, type:

```
start
sysbus.cpu PC
sysbus.cpu PC
```

If the second `sysbus.cpu PC` prints a different value than the first, your environment is healthy. Type `quit` to exit.

If anything above fails, see **§ 5 Troubleshooting**.

3. The nine exercises

Each lab has its own detailed `README.md` with a 7-section walkthrough — bring up, what just happened, first commands, headless UART recipes, useful monitor command tables, mini- experiments, and clean exit.

Lab	Time	What you'll do	Detailed README
00	~5 min	A 5-minute MMIO warm-up: bare-metal ARM Cortex-A9 reads/ writes four 32-bit registers in a memory-mapped "SmartTimer" stub, then you read them back from the monitor. (Aliased: `lab demo`.)	`labs/00-Demo/README.md`
01	~20 min	Boot a bundled Contiki firmware on a simulated STM32F4 Discovery board. Read UART, single-step the CPU, blink the on-board LED from the simulator.	`labs/01-bundled-stm32f4/README.md`
02	~30 min	Boot an unmodified RISC-V Linux kernel (5 cores, OpenSBI, BusyBox userspace) on the SiFive HiFive Unleashed model. Poke around `/proc`, trace UART traffic at the bus level.	`labs/02-linux-on-hifive/README.md`
03	~45 min	Cross-compile bare-metal C for RV64. Run it on a 9-line custom SoC you can edit. Add a second peripheral with one line.	`labs/03-custom-soc/README.md`

Lab	Time	What you'll do	Detailed README
04	~45 min	Bare-metal on a **real** SiFive FE310 (HiFive1). Drive the SiFive UART and GPIO at their datasheet addresses; blink an LED on RV32.	`labs/04-sifive-fe310/README.md`
05	~60 min	RISC-V machine-mode interrupts: program `mtvec` / `mie` / `mstatus` and the CLINT timer, then blink the LED from an interrupt handler.	`labs/05-fe310-interrupts/README.md`
06	~45 min	Headless CI: write a Robot Framework suite that boots firmware, asserts UART output, and fails the build (non-zero exit) on regressions.	`labs/06-robot-testing/README.md`
07	~75 min	Model your own peripheral: write a memory-mapped timer IP in C#, compiled by Renode at runtime, that raises an interrupt the firmware handles.	`labs/07-custom-peripheral/README.md`
08	~45 min	*(optional)* Multi-node IoT network: boot three FE310 machines in one emulation, wire their `uart1` s onto a shared `UARTHub` , and watch two sensor nodes' reports converge on a gateway.	`labs/08-multi-node-iot/README.md`

Do them **in order** — they increase in difficulty. Lab 00 is a 5-minute sanity check that the toolchain works; 01–02 then run bundled images, 03 builds a minimal custom SoC, 04–05 move to a real SiFive chip with real peripherals and interrupts, 06 turns it all into an automated regression test, and 07 has you write a brand-new peripheral model that the CPU talks to. Each one introduces a concept the next assumes (the three Renode primitives `mach create` , `LoadPlatformDescription` , `LoadELF` + `start` ; then real peripherals, interrupts, and automated testing). Lab 08 is **optional** — a multi-node capstone that runs several machines at once and connects them over a shared bus.

4. Where your edits live

The `lab NN` command does not run the lab from `/labs/...` directly. On first invocation it copies the lab into a **writable scratch tree** under your home directory and runs it from there:

Path	What it is	Survives Codespace stop?	Survives container rebuild?	Survives Codespace delete?
`/labs/<lab-name>/`	Read-only image content	yes	no	no
`~/work/<lab-name>/`	Editable copy made by `lab NN`	**yes**	no	no
`/workspaces/renode-lab/`	The cloned git repo	yes	yes	only if `git push`-ed

Practical rules:

- If you just want to tweak a `.resc` or a `hello.c` and re-run, edit `~/work/<lab-name>/...`. The next `lab NN` reuses your edits (`cp -ru` is idempotent — it never overwrites existing files).
- If you want changes to **outlive Codespace deletion**, you must push them to a git remote you own — see § 4.1 below. **A pull request is *not* required just to keep your work**; it's only needed if you want your changes merged into upstream.

4.1 Persisting your work past Codespace deletion

`prag79/renode-lab` is read-only for you (only the maintainer can push). To save your edits, fork the repo and push to your fork. Run these commands **inside the Codespace**, in the cloned repo at `/workspaces/renode-lab/`. Step 1 is one-time; steps 2–4 are the loop you'll run any time you want to checkpoint progress.

```
cd /workspaces/renode-lab
```

1. One-time: fork on GitHub and rewire the remotes. The `gh repo fork --remote` flag automatically renames the existing `origin` (which points at `prag79/renode-lab`) to `upstream` and adds your fork as the new `origin`:

```
gh repo fork --remote --remote-name=origin
git remote -v       # sanity check:
                    #   origin    https://github.com/<your-username>/renode-lab.git
                    #   upstream  https://github.com/prag79/renode-lab.git
```

> **If `gh: command not found`** — your Codespace is running an older image that doesn't have the GitHub CLI baked in. Either **rebuild the Codespace** (palette → `Codespaces: Rebuild Container`) to pull the latest image, which includes `gh`, or install it by hand:
>
> ```
> sudo mkdir -p -m 755 /etc/apt/keyrings \
>  && wget -nv -O- https://cli.github.com/packages/githubcli-archive-keyring.gpg \
> ```

> ```
>     | sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null \
>   && sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg \
>   && echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-archive-keyring.gpg] https://cli.github.com/packages stable main" \
>     | sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null \
>   && sudo apt-get update && sudo apt-get install -y gh
> ```
>
> The hand-install survives Codespace stop/start but is wiped on rebuild — prefer rebuilding if you can.

2. (Recommended) work on a branch, not on `main`. This keeps your tinkering separate from upstream so future `git pull upstream main` is conflict-free:

```
git checkout -b my-experiments
```

3. Copy your edits from `~/work/...` into the git-tracked lab folders inside the cloned repo — not into `/labs/...` at the filesystem root (that path is read-only image content and is not in git):

```
cp -ru ~/work/00-Demo/.       /workspaces/renode-lab/labs/00-Demo/
cp -ru ~/work/03-custom-soc/. /workspaces/renode-lab/labs/03-custom-soc/
# ...repeat for any other lab you tweaked.
```

> **Two different `labs` paths — don't mix them up:** - `/labs/00-Demo/` — baked into the Docker image, read-only, **not** in git. - `/workspaces/renode-lab/labs/00-Demo/` — inside your cloned repo, **tracked by git**. Copy here before `git commit`.

4. Commit and push to your fork.

```
git add -A
git status                                           # review what changed
git commit -m "lab 03: my UART experiments"
git push -u origin my-experiments                    # pushes to <you>/renode-lab
```

Visit `https://github.com/<your-username>/renode-lab/tree/my-experiments` and confirm your files are there. The Codespace itself is now disposable: `gh codespace delete -c <name>` is safe.

Get your work back on a fresh laptop / new Codespace. Open a new Codespace **from your fork** (on `<you>/renode-lab` click *Code* ⟶ *Codespaces* ⟶ *Create on `my-experiments`*) and your edits are already in `/workspaces/renode-lab/`.

Optional — contribute upstream. If you fixed a typo or want to share a lab improvement with everyone, open a PR:

```
gh pr create --repo prag79/renode-lab \
    --base main --head "$(gh api user -q .login):my-experiments" \
    --title "lab 03: my UART experiments" \
    --body  "Short description of what changed and why."
```

5. Troubleshooting

<table>
<tr><th>Symptom</th><th>Likely cause</th><th>Fix</th></tr>
<tr><td>`lab 01` prompt appears but `sysbus.cpu PC` keeps returning `0x00000000`</td><td>You haven’t typed `start` yet — the bundled `.resc` loads the ELF but doesn’t release the CPU from reset</td><td>Type `start` once.</td></tr>
<tr><td>The terminal is unreadable: UART log lines stream non-stop and your typing doesn’t echo</td><td>UART output is being mirrored to the console at INFO level</td><td>`pause` (type blindly, then **Enter**), then `logLevel 3 sysbus.uart4`, then `start`.</td></tr>
<tr><td>noVNC tab shows “Failed to connect” or HTTP 502</td><td>Port 6080’s WebSocket handshake hadn’t completed when the tab opened</td><td>Wait 10 s and click **Connect** in the noVNC page. If still broken: in VS Code’s **Ports** panel, right-click port 6080 ⟶ **Port Visibility** ⟶ **Public** (or sign in to the popup).</td></tr>
<tr><td>Browser tab opens but shows the noVNC welcome screen, not a desktop</td><td>URL is missing the path</td><td>Append `/vnc.html` to the URL, then **Connect**.</td></tr>
<tr><td>`pgrep -af 'Xvfb\|fluxbox\|x11vnc\|websockify'` shows fewer than four processes</td><td>The headless desktop didn’t fully start</td><td>Run `/usr/local/bin/entrypoint.sh true`. Check the per-service logs in `/tmp/Xvfb_:1.log`, `/tmp/fluxbox.log`, `/tmp/x11vnc_-display_:1.log`, `/tmp/websockify.log`.</td></tr>
<tr><td>`lab list` stops at **07** (no **08**) after rebuild</td><td>Codespaces reused a cached `ghcr.io/...:latest` image even though CI pushed a newer one</td><td>`cd /workspaces/renode-lab && git pull`, then **Rebuild Container** (postStart now runs `sync-labs.sh` to overlay `lab` + `labs/` from the repo). Or run `bash sync-labs.sh` by hand after `git pull`.</td></tr>
<tr><td>`lab` does nothing or says command not found</td><td>You’re in a shell where `/usr/local/bin` isn’t on `$PATH` (rare)</td><td>Run it with the full path: `/usr/local/bin/lab list`.</td></tr>
<tr><td>Codespace fails to start with “Failed to pull image”</td><td>GHCR rate-limited you or the image package is private</td><td>Visit https://github.com/prag79?tab=packages ⟶ `renode-lab` ⟶ check it’s **Public**. Retry.</td></tr>
<tr><td>You changed something under `/labs/...`, ran `lab NN`, and your change had no effect</td><td>Edits to `/labs/...` are overlaid on a read-only image and discarded on container rebuild</td><td>Edit the file under `~/work/<lab-name>/...` instead, then `lab NN` again.</td></tr>
</table>

Symptom	Likely cause	Fix
	— the dispatcher uses `~/work/<lab-name>/...`	

6. Cost discipline (so you don't burn quota)

GitHub will not charge you anything as long as you stay inside the free tier. Two habits keep you there:

1. **Stop the Codespace when you walk away.** In the bottom-left of VS Code (or the **Codespaces** menu in your GitHub profile) click **Stop codespace**. Compute billing pauses immediately. Storage keeps ticking at ~$0.07 / GB / month — tiny, but cumulative.
2. **Delete the Codespace when you're done for the day** — but only after committing or copying out anything you want to keep. From your laptop terminal:

```
gh codespace list                                    # find the name
gh codespace delete --codespace <name> --force
```

   ...or use the web UI at https://github.com/codespaces.

You can always recreate a Codespace from the badge in the README. The first launch is the only slow one (~60 s); after that it's instant.

7. Where to go after the labs

- The Renode user docs: https://renode.readthedocs.io/
- Renode's bundled platform `.repl` files for ~100 other boards: `/opt/renode/platforms/` inside your Codespace
- The Antmicro YouTube channel for SoC-modelling walkthroughs.

If you find a bug or have an improvement, open a PR against https://github.com/prag79/renode-lab. The full toolchain (lab content ⟶ Docker image ⟶ GHCR ⟶ Codespaces) is in this one repository and welcomes contributions.

Quick reference card (print this if you want a single-page cheat sheet):

```
Open lab:           click the badge in the README
List labs:          lab list                              (banner runs this on attach)
Quick taste:        lab 00      (a.k.a. lab demo)         5-min ARM MMIO warm-up
Optional capstone:  lab 08                                  multi-node IoT (3x FE310)
Verify install:     lab list && lab 01 -> start -> sysbus.cpu PC -> quit
Edit + re-run:      edit ~/work/<lab>/..., then lab NN
Read UART:          (in monitor) sysbus.uartN CreateFileBackend @/tmp/uartN.log true
                    (in another terminal) tail -f /tmp/uartN.log
Quiet log spam:     logLevel 3 sysbus.uartN
Pause/resume CPU:   pause / start
Read PC:            sysbus.cpu PC
Step one insn:      sysbus.cpu Step
Reset firmware:     machine Reset
Exit Renode:        quit
Save your work:     gh repo fork --remote --remote-name=origin    (one-time)
                    git checkout -b my-experiments
                    cp -ru ~/work/<lab>/. /workspaces/renode-lab/labs/<lab>/   (per
lab; NOT /labs/)
                    git add -A && git commit -m "..." && git push -u origin my-
experiments
Stop Codespace:     bottom-left of VS Code -> Stop codespace
```

```
Delete Codespace:   gh codespace delete -c <name> --force         (safe AFTER push
above)
```